

Physics at Play: PPO-Based Gameplay for Generalizable Physics-Aware Reinforcement Learning

Trisha Bhatawdekar
University of Washington

Jules Ropars
University of Washington

Daniel Shubin
University of Washington

Abstract

Proximal Policy Optimization (PPO) is a reinforcement learning algorithm used to train AI models by, as the name suggests, optimizing policies. It addresses a key downside of policy gradient methods, which are sensitive to step size, by constraining each update to the previous policy such that the cost function is simultaneously minimized and the objective is improved. A traditional PPO involves two neural networks that play different roles. One of the networks is an actor and determines how the agent acts in a given state. The other is a critic, and its main goal is to evaluate the current state by considering the potential future rewards, thus helping to guide the actor network. In this paper, we aim to leverage PPO in a physics-based Bonk.io environment to investigate whether models can learn and understand physics through gameplay. Furthermore, by comparing a baseline PPO model with a curiosity-based model and an auxiliary prediction-based model, we seek to determine whether reward-based or loss-based feedback will influence the ways in which the models learn physics-based controls. Through this comparison, we hope to better understand how different rewards/losses can influence the efficiency / generalization of each model as they learn behaviors such as movement, collisions, and strategy in a physics-based environment.

1. Introduction

Reinforcement learning (RL) has seen success in games because games provide clear states, actions, rewards, and continuous, consistent feedback. Simple games such as Tetris, Space Invaders, and Moon Lander excel at this simple loop, but more advanced modern games propose new challenges. Physics is a notable obstacle, as when departing from grid-based movement and deterministic results, an RL model is suddenly faced with a near-infinite continuous array of outcomes to a given action. Physics-based gameplay also offers a compelling case for model evaluation,

as agents must demonstrate meaningful physical strategies over memorized button sequences.

Bonk.io is a 2D multiplayer physics-based online game in which players attempt to push opponents off the map while simultaneously trying to avoid being pushed off themselves. The game’s mechanics, which include movement (e.g., pushing and bouncing), real-world gravity scaling, inertial mass, and collisions, make it an ideal, challenging environment to train a reinforcement learning model. The simplicity of the game, paired with a depth in strategy and timing, provides an ample environment to train an RL model.

As Bonk.io is not open-source, we recreate a clone locally, using video tracking and comparing observed vs simulated motion to match the game’s physics parameters and game feel. Then, turning the game’s state into observation vectors, we feed them to three different modified PPO models and pretrain with two different variants. First, we train across a diverse, increasingly difficult five step curriculum, progressing when the current map is mastered (a win rate of 70%). Secondly, we create a single complex “playground” map that contains more different obstacles and examine if a curiosity model can outperform the other models. The goal for this project is to (1) determine whether an RL agent can effectively learn physics through gameplay and (2) compare how generalizable the three models are when fine-tuning on new maps.

2. Related Works

Reinforcement Learning has taken great strides over the past few years, and though no prior work has trained an RL model to play Bonk specifically, there are many papers discussing RL in physics environments. For our project, we focus three are highly relevant works. First, Ramesh et al. [3] introduce the concept of using a PINN (physics-informed neural network) based model to train an RL algorithm, and demonstrates that, in challenging environments,

a model-based RL algorithm performs better, on average, than a model-free RL algorithm. Building off of this idea, Lin et al. [1] discusses how having an adaptive auxiliary task for an RL model’s NN loss function

$$(L(\theta) = L_{main}(\theta) + X * sum(w * L_{aux}(\theta)))$$

could help the model learn efficiently. Finally, Merckling et.al [4] paper, takes a different approach by rewarding curiosity to encourage agents to explore uncertain scenarios, thus illustrating how this strategy makes RL training process more efficient. These three papers form the basis of our paper, and thus, the three models we compare. We combine ideas from Lin et al. [1] and Ramesh et al. [3] to examine how using a PINN as an auxiliary task can help improve the model’s training accuracy and efficiency. Moreover, we examine how using a PINN as our neural network and updating the respective loss function, compares to rewarding curiosity (Merckling) and how it can affect effectiveness when learning physics-based controls.

3. Methods

3.1. Environment

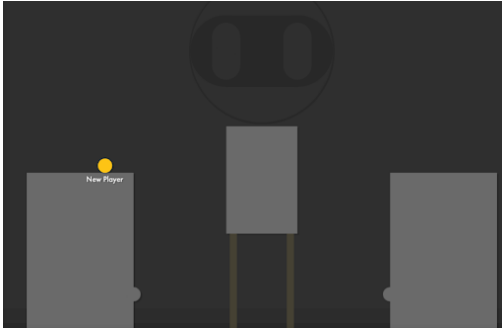


Figure 1. Original Bonk.io environment

Since the code for Bonk.io is not open source, we recreated a local version of the game environment. The original game uses Box2D, a physics engine, as it is meant to be a realistic physics simulation. We leverage approximations of the in-game environment parameters (e.g. object friction, bounciness, scale, relative x and y position, etc.) to create a to-scale Bonk.io replica. The main challenge in recreating the game is from matching the magnitude of each action impulse combined relative to the gravity. Using Tracker, an open source physics video tracker tool, we compare our recreated game with the original, and match the expected and simulated player velocities/accelerations. Lastly, to help the model generalize better, rather than just memorizing specific map configurations, we recreated some of the original Bonk.io maps (Gang Grounds, Grapple Battle, etc.) to scale. From this recreated Bonk environment,

we extract each object’s (e.g. platforms, player 1, player 2) locations, velocity, and acceleration, and input them to the RL model as an observation vector.

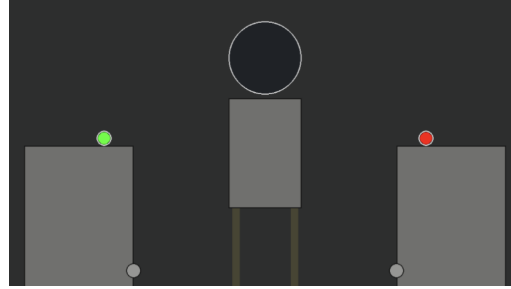


Figure 2. Our recreated Bonk.io Environment

3.2. Pretraining Method: Curriculum

We spread the pretraining process into a five-step curriculum, moving to the next stage upon mastery of the current step. Each stage introduces a new challenge through different map elements to interact with. The model advances to the next stage once it has demonstrated that it can consistently claim the objective. We define the curriculum as follows:

- **Stage 1: Tutorial Flat Objective**

- The first stage focuses on claiming the objective on a flat map without obstacles. This is a fairly straightforward stage, as it allows the model to learn basic movement skills and positioning, prior to introducing more complex obstacles and opponents.
- The map used is a replica of one of the most basic (and popular) maps in Bonk.io.

- **Stage 2: Tutorial Gap Jump**

- The second stage ramps up in difficulty and involves maneuvering around obstacles on a flat map with holes and walls obstructing the objective. This enables the model to practice avoiding obstacles and practice navigating the map.
- Additionally, it teaches the model to avoid out of bounds and to jump towards the objective.

- **Stage 3: Moving Objective Court**

- The third stage consists of a left/right-oscillating capture zone platform high up in the air. To reach the objective, the model must use the bounce pad and track the moving platform.
- This map introduces feature use via bounce pads and teaches the model to leverage momentum. It also includes a non-stationary objective that the model must track.

- **Stage 4: Procedural Parkour**

- The fourth stage introduces a pyramid-like ladder of platforms that slightly adjusts over time. The player spawn points alternate between left and right, and the objective remains in the center. Every run, the ladder rungs (platforms) also rise/fall slightly to ensure that the model does not overfit and simply memorize a single solution. We also introduce death zones that the model must learn to avoid.
- Throughout training on this map, we also reintroduce the maps from the previous stages, ensuring that the model does not forget what it has previously learned.

- **Stage 5: Map Cycle**

- For the final stage, we cycle through various maps (both learned and new). We also introduce new maps like High Low Duel, a map where we start on a map similar to the one in [Figure 2] but the capture zone platform is lower down, and other maps that are variations of the previous stages. This is the culmination of the pretrain step in which we give the model a taste of everything it has learned.

3.3. Pretraining Method: Playground

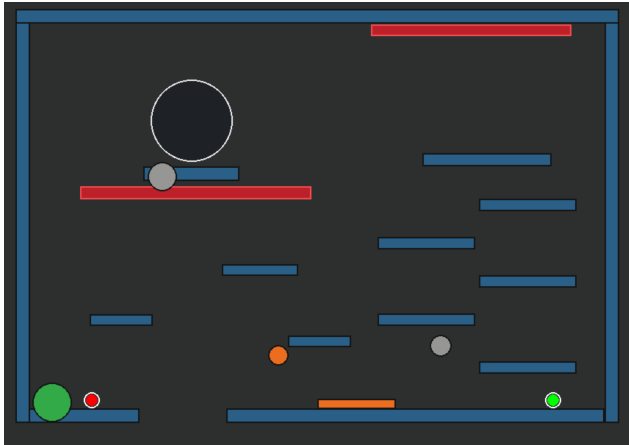


Figure 3. The Playground Map

In addition to our curriculum strategy, we employ a playground pretraining method in which the model is placed in a single map for the entirety of its training cycle. The map is built to contain a wide variety of Bonk.io mechanics and obstacles, including multiple routes to the objective, death zones, different types of bounce pads, and parkour platforms. Rather than simply exposing the model to a series of increasingly difficult maps, as in curriculum training, Playground aims to provide a compact yet comprehensive environment in which the model can train, allowing us to observe if the model can learn generalizable Bonk.io movement from a single sufficiently diverse environment,

and if curiosity or physics-informed loss has a bearing on its ability to do so. If the map includes enough platform layouts and a variety of obstacles, the model may be able to learn all relevant mechanics of the game needed for the curriculum, but only from a single map. In this way, the playground functions as a dense environment for end-to-end training, where the objective is not to master any single aspect of movement, but to repeatedly encounter a broad array of situations and figure out approaches to handle them dynamically. This design is inspired by prior work showing that diverse environments can encourage the emergence of robust control behaviors without requiring a separate reward term for every individual technique/skill [2].

3.4. Fine-Tuning Method: Six Stages

After pretraining the six models, we observe how well they generalize to new maps. To analyze results, we fine-tune the model on six completely new maps to see how quickly the models can learn. For each map, we fine-tune each model until it achieves a win-rate of 85% (or is capped at 10M steps) and analyze win-rate / distance to objective to compare. The six new maps are as follows:

- **Map 1: Validation Combat Fight**

- In this map, we pit the models against a basic scripted opponent to observe how well the model learns collisions and dueling. This is the sole-validation map in which we have an opponent. All other maps have idle, non-interactive opponents.

- **Map 2: Validation Safe Ladder Climb**

- This map, which is a callback to Map 4 of the pretraining step, features a vertical ladder where the model must leverage, while alternating from jumping left to right, to reach the objective.

- **Map 3: Validation Death Gap Parkour**

- This map requires the model to learn navigate a parkour-structure that contains gaps. If the models, falls through a gap, it will die and lose the current progress it has made towards the objective. This map ultimately reintroduces death zones so that the model relearns avoidance of gaps due to the penalty of death.

- **Map 4: Validation Moving Ladder Climb**

- The map is similar to Map 2, however, two of the platforms are no longer stationary but rather oscillate left and right. This tests each model’s ability to transfer the platform tracking ability it learned on Stage 3 of pretraining.

- **Map 5: Validation Moving Death Parkour**
 - This fifth map is a combination of Maps 2, 3, and 4, and combines the reintroduction of death zones with the reintroduction of moving platforms.
- **Map 6: Validation Bounce Arc Trial**
 - This map is different from the others in the validation set as it is designed to assess how quickly the model adapts to feature use. It incorporates several bounce pads that the model must leverage to reach the objective. The bounce pads are also separated by two death zones such that the model must learn to control its momentum while bouncing.

3.5. Basic PPO Model

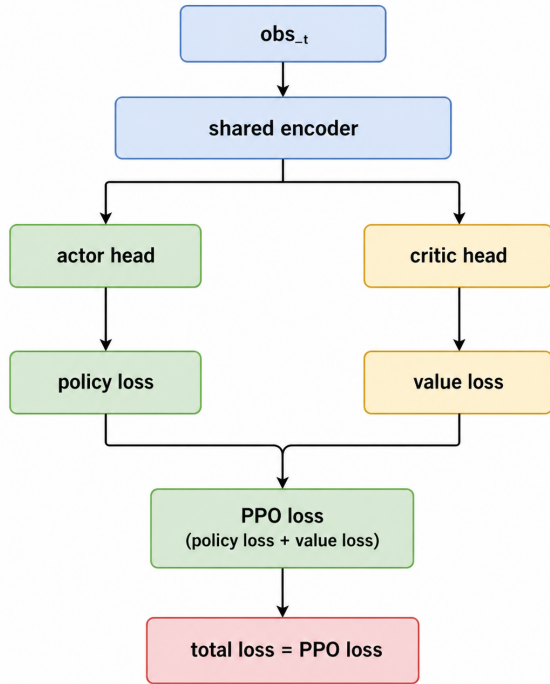


Figure 4. Architecture Overview of Base PPO Model

Our baseline PPO model was implemented using the Stable Baseline3 reinforcement learning library. Using this library enabled us to focus on the specific PPO model specializations and avoid reinventing the wheel. The baseline model uses the actor and critic neural networks as hinted at earlier [6]. As the agent interacts with the environment’s stimuli and traverses the map, it converts the current state into an input vector. This input (obs_t) [Figure 4] contains data from 16 ray casts as well as information like nearest platforms, capture zone location, current x/y position, and

more. This data is encoded and then passed to the actor model which determines which actions to take. Simultaneously, the value network determines the value of the current action by evaluating the potential for future reward. Stable Baseline3’s implementation of PPO normalizes advantage and uses clipping for the value function. By keeping the policy updates constrained, the model updates such that it does not end up in an irreversible bad state, thus keeping the learning process fairly stable. For our experiment, the PPO model provides a starting point with which to build off to see how curiosity-based rewards or loss-based feedback affect the way that PPO models learn physics-based controls such as momentum, velocity, positioning, collisions, and more. It is important to highlight that PPO is based on the following loss equation:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

Where $r(\theta)$ is the ratio between the old and new policy. Overall, we use this basic model to establish an expected performance benchmark which we can compare the other two architectures to.

3.6. Modified Architecture 1 - Auxiliary-Based

The concept behind this model is based on a combination of Lin et al.’s adaptive auxiliary task weighing framework [1], Ramesh et al.’s physics-informed model-based reinforcement learning based approach [3], and recent work on physics-informed policy optimization through analytic dynamics regularization [5]. As discussed before, the existing logic for the basic PPO model contains two models, the actor and critic. We build upon this architecture in the auxiliary-based model and add an additional physics auxiliary head. Given the current observation and the action at time step t , the head predicts the next observation (i.e., the player’s next x/y position as well as velocities in the x/y directions). We use a shared latent (shared encoder) to learn how to encode the observation (modified from before, to now include current x/y velocity) across the actor, the critic, and the auxiliary model.

To achieve this, we override the existing PPO rollout method to output the next observation (used during the loss computation) and create a new simple linear neural network with a ReLU activation function to predict these physical quantities. Lastly, and most importantly, we update the existing PPO loss function and combine it with the auxiliary loss function discussed previously

$$L(\theta) = L_{\text{main}}(\theta) + \lambda_t L_{\text{aux}}(\theta) \quad [1]$$

and adding the physics loss term to the current loss:

$$L(\theta) = L_{\text{PPO}}(\theta) + \lambda_t (L_{\text{PRED}}(\theta) + L_{\text{PHYSICS}}(\theta))$$

where

$$L_{\text{PHYSICS}}(\theta) = \|\hat{\mathbf{p}}_{t+1} - (\mathbf{p}_t + \mathbf{v}_t \Delta t)\|^2$$

with

$$\mathbf{p}_t = (x_t, y_t), \quad \mathbf{v}_t = (v_t^x, v_t^y)$$

and

$$L_{\text{PRED}}(\theta) = \text{MSE}(\hat{x}_{t+1}, x_{t+1}) + \text{MSE}(\hat{y}_{t+1}, y_{t+1}) \\ + \text{MSE}(\hat{v}_{t+1}^x, v_{t+1}^x) + \text{MSE}(\hat{v}_{t+1}^y, v_{t+1}^y)$$

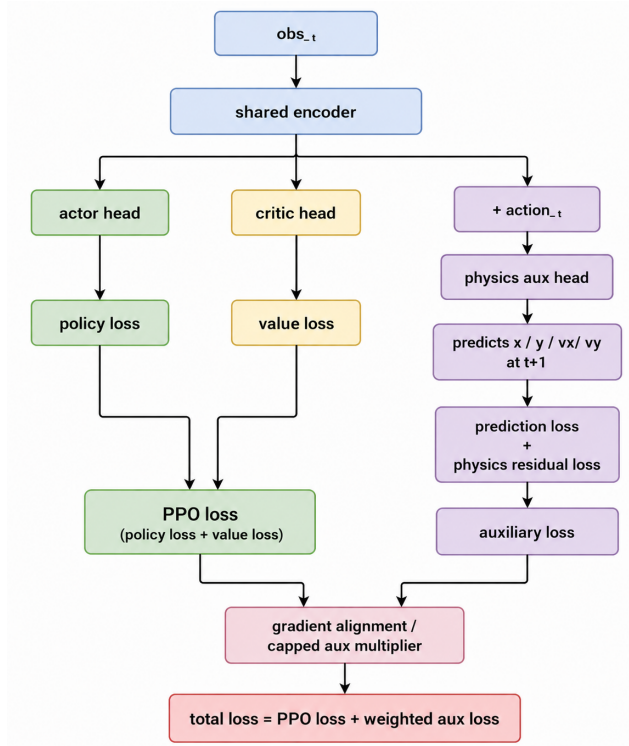


Figure 5. Architecture overview of base PPO model modified with a physics-based auxiliary task.

It is important to note that we set L_{PHYSICS} using OL-AUX-5 [1] since we want it to learn the value of PHYSICS based on the inner product of the gradients, and we include $L_{\text{PRED}}(\theta)$ to prevent the model from predicting physical quantities of 0. This results in a PINN-like loss function where we check whether the model is able to take an observation and accurately predict the laws of physics.

The purpose of making an architecture like this is to confirm that (1) having an auxiliary task helps the main model decrease its own loss [1] and (2) that a physics-informed model is able to achieve better and more efficient returns compared to a basic model [3].

3.7. Modified Architecture 2 - Curiosity-Based

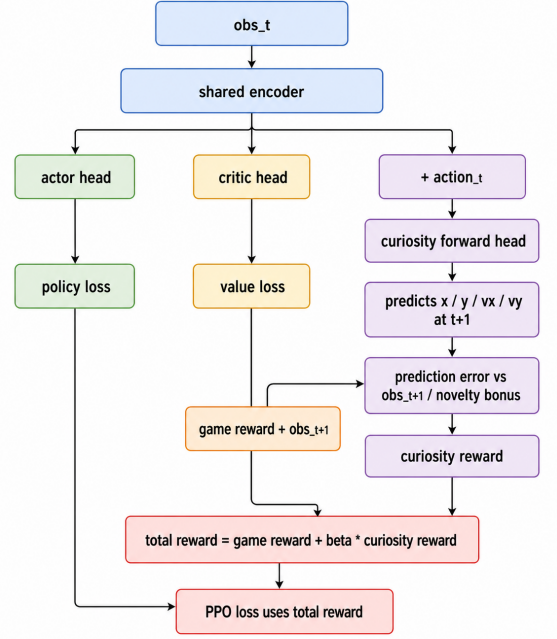


Figure 6. Architecture overview of base PPO model modified with a physics-based curiosity reward bonus.

This architecture also maintains the same basic PPO actor-critic structure as the baseline. The main difference is that instead of a pure resulting-environment derived reward system, we add a curiosity module that rewards the agent for encountering movements it cannot yet predict well. By directly tying reward to new experiences, we incentivize the model to explore its training environment and find new physics interactions independently, without needing a strict curriculum. In our Bonk.io environment, this should result in a model that proactively seeks states dealing with bounces, collisions, edge recovery, and opponent interactions, rather than just optimizing for the immediate game objective.

In the auxiliary-based model, prediction error is added to the model’s loss function, pushing the network towards a more physics-aware representation. In the curiosity-based model, we do not use prediction error for PPO loss, but instead sum it with the reward function during rollout:

$$r_t^{\text{total}} = r_t^{\text{game}} + \beta * r_t^{\text{curiosity}}$$

Here, r_t^{game} is the normal Bonk.io reward, $r_t^{\text{curiosity}}$ is derived from the model’s next-state prediction error, and β is how intensely curiosity affects the total reward. Unknown or poorly understood action results create larger prediction errors, which in turn boost the reward, leading

to a model that actively explores to find new experiences in its environment. Once the agent learns the results of these actions, the error drops, and the same behavior is no longer rewarded. This shifts the goal of the model from a consistent understanding of physics to exploration.

This architecture creates an interesting comparison between the two training configurations of our project. When undergoing curriculum training, curiosity might help the agent discover relevant behaviors faster at each stage. In the playground pretraining, curiosity can become significantly more important, as it drives the model to learn every unique feature (bounce pads, drops, etc) in the playground map, ensuring it learns everything possible before moving on.

4. Experiments

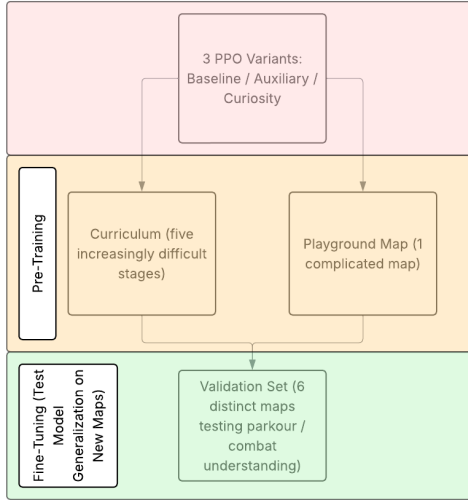


Figure 7. Flowchart depicting the experimental setup (from model to pretraining to finetuning)

We compare six different models. Specifically, we pretrain all three architectures in (1) a five-step curriculum of increasingly difficult maps where the model progresses upon mastery of the current stage or (2) a complex “playground” map. With the six pretrained models, we present five new maps where we test each model’s learned physics-aware behavior through combat and parkour ability. Over a maximum of 10M steps, we fine-tune each model and compare how quickly each is able to reach a win rate of 85%. With this setup, we observe how generalizable each pretrained model is. At this point, there are two likely outcomes (1) that Bonk is a simple enough game where the baseline PPO model is on par with the two modified models or (2) that the game is complex enough

where both the auxiliary and curiosity models are more efficient and are able to tune on newly seen maps faster. We hypothesize the latter. Moreover, we hypothesize that curriculum-trained auxiliary and curiosity outperform their respective playground-trained counterparts. We expect curiosity curriculum to be the optimal model as it encourages exploration and environment interaction.

After running the pretraining process for these six variants, we output a CSV file containing timesteps, steps per second, success rate, episode reward, loss, and the physics aux multiplier (for the auxiliary model). We also save checkpoints for each model, which we later fine-tune on the five new maps to record observations.

5. Discussion

Our results do not provide a clear ‘best’ architecture. The modified PPO model’s performance relative to baseline heavily depends on both the training regime and evaluation map. The baseline playground model performed the worst on the transfer evaluation maps, suggesting that PPO cannot sufficiently generalize to new environments when trained only on a single complex map and instead overfits. Interestingly, the curriculum-trained baseline model outperformed many of the modified PPO models during transfer tests. The auxiliary and curiosity models displayed clear advantages in the pretrained model duels, and a higher ability to generalize when given less starting diversity of information (Playground). These results, unfortunately, did not transfer equally to all fine-tuning maps. Our primary result is that curiosity and auxiliary architectures do not universally improve PPO performance, but do provide the agent with behavior better suited to extracting information from environments without explicit guidance (curriculum)

5.1. Pretrain Baseline Evaluation (Playground)

The model was trained on a single playground map containing the key map elements, such as moving floating platforms and bouncepads. Since this map contained a wide variety of interactive elements, it served as a useful starting point to assess whether the baseline PPO model could learn basic physics feedback. Additionally, the model was trained primarily using self-play so that it could improve by playing against itself as an opponent.

Overall, the baseline PPO model trained on the playground was able to master the playground and consistently capture the object before 10M total steps.

However, there were some limitations. Since the baseline PPO model was trained on a single playground map, it ended up learning map-specific tricks to reach the objective.

For instance, it learned to exploit a bouncepad trick that would propel the player up to the objective. However, this specific skill is not generalizable. Thus, when the model was evaluated on other maps, especially those with gaps, it would perform poorly. This suggests that training the model for 10M steps on the playground likely led to overfitting to the map.

5.2. Pretrain Baseline Evaluation (Curriculum)

In contrast, the baseline PPO trained on the curriculum was able to generalize better since it was trained on a variety of maps. Still, it generally performed poorly. We hypothesize that 10M steps across every stage combined were likely not sufficient for the model to grasp the map and stabilize its performance. Curriculum learning usually requires enough time for the model to master earlier stages so it can succeed in later stages. With these baseline results established, we proceed to discuss and compare the results for the curiosity and auxiliary-based models.

5.3. Pretrain Duel: Baseline vs Auxiliary

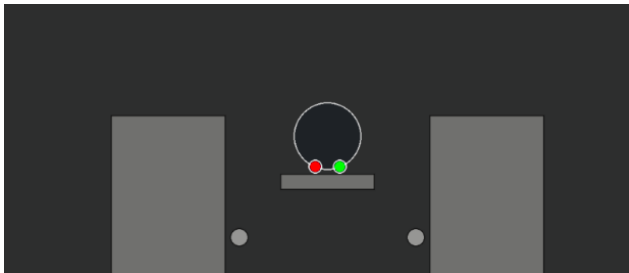


Figure 8. We used this map (tutorial gap jump) to host duels between the three models. The map contains basic platforms, jump pads, and a capture zone.

For both the playground-trained and curriculum-trained models, the auxiliary outperformed its respective baseline opponent. Running the duel for 10 rounds each, the auxiliary model won 10/10 both times. While the playground baseline put up a fight, this suggests that even after 10M training steps, the baseline curriculum model was still struggling to play against an opponent. Specifically, the auxiliary model was able to push off the opponent / capture the zone within a few seconds regardless of initial spawn.

5.4. Pretrain Duel: Baseline vs Curisioity

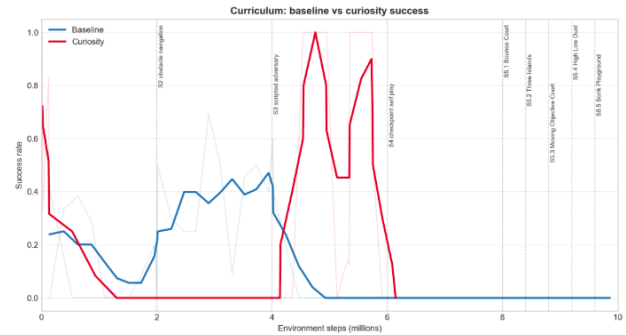


Figure 9. This graph compares the success rate for the curriculum-trained baseline and curiosity model.

It is interesting to look at the success rate of baseline and curiosity for the different curriculum stages (Fig 9). The baseline has some initial success in the early stages, even steadily improving until the end of the obstacle navigation phase. But, once it hits the scripted adversary, it quickly plummets and drops to a 0% success rate, unable to overcome an adversary that fights back. Curiosity experiences almost the opposite, quickly hitting a high success rate in the first stage, it abandons the goal and instead focuses on maximizing its curiosity-based reward, jumping and moving in new ways. This happens due to the diminishing reward from repeated actions, primarily designed to incentivize the model to seek out new behaviors and strategies, but it also unfortunately pushes it away from existing winning methods.

This works to curiosity’s advantage, though, when it hits the scripted adversary stage. Where baseline PPO fails, curiosity is able to quickly adapt to this new opponent and environment, driven by the wide range of emerging strategies now possible with an active second player. We can see from the two spikes in this stage of the graph that the curiosity curriculum model found two winning strategies against the scripted bot, each time dropping them once perfected to seek out new, more rewarding approaches.

This unfortunately shows the main flaw in the curiosity approach, that while the model will find winning strategies much quicker thanks to its exploration mindset, it will also actively avoid them once repeated a few too many times due to the dwindling novelty reward bonus, and will usually finish training in a state halfway to a new strategy, and perform poorly against other models that stuck with their strategy once mastered. This explains the head-to-head results of curiosity against baseline, which was 20% win rate with playground training, and 100% in

curriculum, only due to baseline’s inability to adapt at all from the scripted opponent onwards.

5.5. Pretrain Duel: Auxiliary vs Curiosity

In test duels between Curriculum Auxiliary and Curriculum Curiosity, the auxiliary model won 63% of the time (tested over 30 rounds). This shows the core difference in the architecture of the two models. Curiosity will endlessly try new strategies, throwing away winning approaches for the sake of new ones due to its diminishing return on exploratory reward for repeated actions. Whereas Auxiliary’s OL-AUX5’s exploration loss weight will quickly drop once it finds a winning strategy, maintaining its performance once learned. This leads to a winning strategy, Auxiliary result consistently defeating a half-baked mid-transition strategy, Curiosity Model.

5.6. Pretrain Auxiliary Multiplier

It’s also interesting to observe the auxiliary multiplier (how much weight the model assigns to the physics auxiliary task compared to the main task loss using OL-AUX5). Looking at the graph below, we see that for the playground, the model learns to weigh the auxiliary multiplier higher. We believe that this makes sense as to lower the loss, it starts to learn to assign a higher weight to physical quantity prediction rather than action loss. Focusing on the curriculum, we also notice an interesting pattern. As the maps are simple, the multiplier stays low. When the stage becomes more challenging (around timestep 500,000), the multiplier suddenly spikes up. Thus, these data points highlight an interesting correlation between environment difficulty and auxiliary task weight.

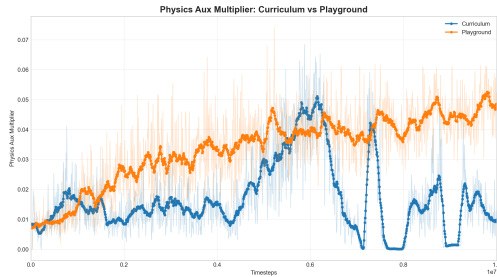


Figure 10. Graph modeling the relationship between the physics-based auxiliary task multiplier vs timestep for both the curriculum and playground trained models.

5.7. Fine-Tuning: Win-Rate

Taking the pretrained model’s, it is interesting to see how well they generalize to new unseen maps. In Figure 11[11], we show three maps (previously described above) where we fine-tuned the six models and observed the win-rate over the course of 10M steps or until a win-rate threshold

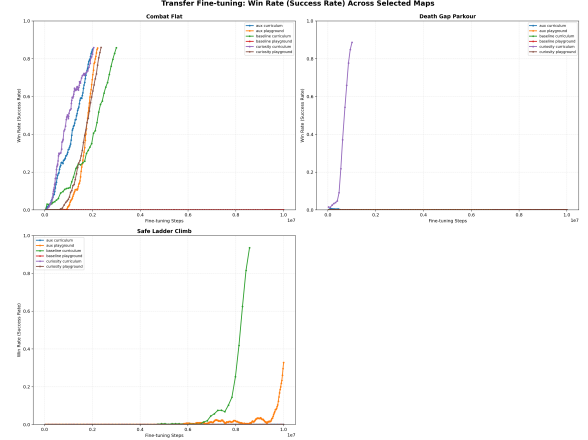


Figure 11. Collection of charts depicting the win-rate across the 10M fine-tuning steps for the three validation maps where the model was able to be successful.

of 85% was reached. Focusing first on the combat flat map, the results seem to match our hypothesis. Both the auxiliary and curiosity model are able to fine-tune faster on this new map than any of the two baseline models. Furthermore, the only model unable to get a single win on this simple map was the baseline playground variant. This makes sense as baseline already takes longer to pretrain and playground is an extremely complex map.

Switching to the remaining two maps, ”Death Gap Parkour” and ”Safe Ladder Climb”, it is clear that they are more difficult to learn in just 10M steps and, most surprisingly, the baseline curriculum in ”Safe Ladder Climb” is able to learn the map quicker than any of the other models though auxiliary playground is not far behind. Likely if we increased the cap from 10M steps to 15-20M the other models would also be able to learn the maps.

5.8. Fine-Tuning: Average Distance to Objective

What can also be used as a more detailed substitute for win-rate is the average player distance to objective throughout the fine-tuning process. Focusing on Figure 12[12], we can observe this relationship for all six validation maps. Most distinctly, it seems that the auxiliary playground model almost always starts poorly, but then rather than spiking up like the other models consistently decreases its loss. Also, across the five maps not including ”Moving death Parkour” the baseline playground performs worst. Furthermore, if a map is difficult to learn, curiosity curriculum outperforms its respective playground model while the opposite is true for the aux model where the auxiliary playground model consistently outperforms the curriculum-trained model. It is important to note that when a line ends early, that is due to the win-rate threshold of 85% being reached and can be



Figure 12. Collection of charts depicting the average distance to objective across the 10M fine-tuning steps for all six validation maps.

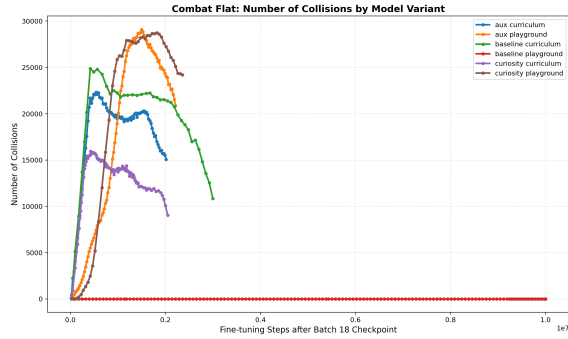


Figure 13. Graph depicting the number of collisions on the duel map throughout the 10M timesteps.

observed in figure 11 as well.

5.9. Fine-Tuning: Number of Collisions During Duel

Lastly, when fine-tuning for dueling, it is interesting to examine how aggressive each model is. We do this by looking at the number of collisions during each duel against a scripted opponent. Looking at Figure 13[13], we can see that initially all three curriculum models are more aggressive than their respective playground models (i.e. that they immediately start hitting their opponents). After a few million steps, however, the aux and curiosity playground models start aggressively hitting their opponents. As they start fine-tuning (and become more successful on the map [11], the number of collisions decreases. This is likely happening from the fact that the model’s learned to collide with more force and require less collisions to push the opponent off the map. Importantly, it seems that while curriculum models learned the main point of this map first, playground models were able to adjust their impact technique faster to

almost make the ground up. We would also like to note that the reason why the baseline playground was unsuccessful on this map, is that it likely overfit on the pretraining steps and was constantly falling off the map.

5.10. Further Research

There are countless extensions to this project that we can consider in the future. First, we can focus more on tuning all the parameters. For this research paper, our primary priority was not to create the best model we possibly could, but examine how three different models, with generally accepted parameters, would perform against each other. Secondly, we can complicate the environment by converting it from 2D to 3D. It would be interesting to learn if the auxiliary/curiosity based models are able to out perform the baseline model in a more complex environment. Additionally, it would be interesting to create a web agent to actually play (and potentially train) on Bonk.io against real human players. These further research topics are just examples, but there are many directions in which this project could go in the future.

5.11. Conclusion

In this study, we examined three different PPO models: the baseline PPO, the curiosity-based PPO, and the auxiliary-based PPO. We hypothesized that both the curiosity-based and auxiliary-based model would outperform the baseline model in terms of both efficiency and accuracy. Specifically, we believed that the auxiliary model would perform best when trained on the curriculum stages while the curiosity model would be ideal for the sole playground map. Ultimately, when running our experiments, our predictions for both efficiency and accuracy were fairly accurate. Both modified models generally outperformed the baseline model while auxiliary performed best through the curriculum training plan. Surprisingly, however, it seems that despite being more adventurous and aggressive, the curiosity model, much like the auxiliary model, performed poorly on the highly complex playground map. Regarding model generalization, in most cases the fine-tuned baseline performed worse than the modified architectures. There were, however, some interesting outliers where the baseline generalized to the new maps faster. This is something that would be interesting to dive deeper into in a future paper.

Through this project, we learned a lot about the field of Reinforcement Learning, as well as the Proximal Policy Optimization algorithm, which still remains state-of-the-art despite its inception nearly a decade ago. We learned about optimizing the model through curiosity rewards and auxiliary-PINN like loss modifiers. While we faced several obstacles and challenges such as recreating the bonk environment, designing good curriculums/maps,

and finding good reward functions, we learned a lot from reading countless papers and from utilizing and optimizing PPO. Through this project, we reinforced our learning.

References

- [1] X. Lin et al., “Adaptive Auxiliary Task Weighting for Reinforcement Learning,” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/0e900ad84f63618452210ab8baae0218-Paper.pdf 2, 4, 5
- [2] N. Heess et al., “Emergence of Locomotion Behaviours in Rich Environments,” 2017. Available: <https://arxiv.org/pdf/1707.02286> 3
- [3] A. Ramesh et al., “Physics-Informed Model-Based Reinforcement Learning,” *Proceedings of Machine Learning Research*, vol. 211, pp. 1–12, 2023. Available: <https://arxiv.org/pdf/2212.02179> 1, 2, 4, 5
- [4] A. Merckling, N. Perrin-Gilbert, A. Coninx, and S. Doncieux, “Exploratory State Representation Learning,” *Frontiers in Robotics and AI*, 2022. Available: <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2022.762051/full> 2
- [5] N. Chandra et al., “Physics-Informed Policy Optimization via Analytic Dynamics Regularization,” 2026. Available: <https://arxiv.org/pdf/2603.14469> 4
- [6] Schulman, John, et al. “Proximal Policy Optimization Algorithms”. 28 Aug. 2017. Available: <https://arxiv.org/pdf/1707.06347> 4